Arkusz sekcji - „Jak to dotyczy naszej aplikacji?”

Arkusz wypełniacie na każdych zajęciach od 2. do 12., zaraz po wprowadzeniu prowadzącego (ok. 10 minut). Punkty 1-4 omawiamy w rundzie z prowadzącym, punkt 5 to wasze zadanie na kolejny tydzień. Wypełniony arkusz zapisujecie w repozytorium jako `docs/arkusze/Z<numer>-<temat>.md`*.*

Nagłówek

Pole	Wartość
Sekcja / aplikacja	Pupilandia - rezerwacje wizyt w salonie groomerskim
Technologia	Flutter (Dart) + Firebase
Zajęcia nr	2
Temat zajęć	Modelowanie zagrożeń i architektura
Obszar MASVS	Fundament: MASVS-AUTH, STORAGE, NETWORK, PLATFORM
Osoby obecne	Jakub Różycki, Mateusz Sarwa, Michał Dyrka

0. Status z poprzedniego tygodnia

Co zrobiliście z zadania z poprzednich zajęć? Każda pozycja wymaga dowodu: link do commita lub pull requesta, zrzut ekranu, wynik testu albo narzędzia.

Zadanie	Status (zrobione / częściowo / nie)	Dowód
Repozytorium GitHub z dostępem dla prowadzącego	zrobione	https://github.com/jrozycki2003/pupilandia_grooming_multiplatform_app
README z finalną kartą projektu	częściowo	Karta jest w docs/karta-projektu.md; w README są linki. Skład 3 os. do zgłoszenia prowadzącemu.
Szkielet aplikacji (build + logowanie Firebase)	zrobione	lib/main.dart (AuthChecker), lib/pages/login.dart; flutter run na Androidzie
Lektura OWASP MASVS	zrobione	Mapowanie ról zespołu na obszary MASVS w karcie

Zadanie	Status (zrobione / częściowo / nie)	Dowód
		projektu
Szkic architektury / DFD	częściowo	Szkic w karcie; DFD v1: docs/threat-model/dfd.md (PNG do dokończenia przez Michała)

1. Gdzie to zagrożenie występuje u nas?

Wskażcie konkretne miejsca: ekrany, funkcje, komponenty, dane z karty projektu. Odpowiedź „nigdzie" wymaga uzasadnienia.

Miejsce w aplikacji	Jakie dane lub funkcja są zagrożone	Klasyfikacja danych
login.dart / signup.dart / forgot_password.dart	E-mail, hasło, token sesji Firebase (SDK)	Poufne
shared_pref.dart (SharedPreferences)	UID, imię, e-mail, URL zdjęcia - jawny tekst; allowBackup nie wyłączone	Poufne
database.dart + kolekcja Booking	Rezerwacje: usługi, termin, cena, groomer, notatki, e-mail. getBookings() może pobrać całą kolekcję	Poufne
home.dart, admin_panel.dart, employee_panel.dart	Role: dokument w Admin/Employee. UI ukrywa panele - to NIE jest ochrona backendu	Poufne
Firestore users + Cloud Storage	Dane konta i zdjęcie profilowe (wizerunek)	Poufne
Mapa salonu + AndroidManifest	Adres salonu publiczny. GPS użytkownika NIE jest zbierany, ale w manifeście są ACCESS_FINE/COARSE_LOCATION	Publiczne (salon). GPS użytkownika byłby szczególnie wrażliwy - obecnie go nie ma
3 przepływy przez granice zaufania (start STRIDE)	1) App -> Firebase Auth (sieć): hasło/token. 2) App -> Firestore Booking/role (sieć). 3) App -> SharedPreferences/backup	Poufne na każdym z trzech przepływów

Miejsce w aplikacji	Jakie dane lub funkcja są zagrożone	Klasyfikacja danych
	(system)	

2. Co już mamy?

Jakie zabezpieczenia w tym obszarze już działają (w kodzie, w konfiguracji, po stronie backendu)?

- • Firebase Authentication (e-mail + hasło), sesja w SDK, authStateChanges() w AuthChecker.
  • Trzy role w UI: klient / pracownik / administrator - dokument w kolekcjach Admin / Employee.
  • Komunikacja z Firebase po HTTPS (SDK); hasła tylko u dostawcy Auth, nie w Firestore.
  • Klucze: lib/api_keys.dart w .gitignore, szablon api_keys.dart.template, SECURITY_SETUP.md.
  • Brak zbierania GPS użytkownika w logice aplikacji (mapa = stały punkt salonu).
  • Reset hasła przez Firebase (forgot_password.dart).
  • Danych szczególnie wrażliwych (zdrowie, biometria, finanse, dokładna lokalizacja użytkownika) obecnie nie zbieramy. Biometria - plan Z4.

3. Czego brakuje?

Luka / zagrożenie	Gdzie	Ryzyko (niskie / średnie / wysokie)	Dlaczego takie ryzyko
Brak Security Rules w repo; role sprawdzane w kliencie	Firestore: Booking, Admin, Employee, users	wysokie	Przy słabych regułach każdy zalogowany czyta cudze rezerwacje albo wpisuje się do Admin
Dane osobowe w SharedPreferences + domyślny backup	shared_pref.dart, manifest	wysokie	Root / kopia zapasowa Androida - wyciek e-maila i UID
Filtrowanie rezerwacji po stronie klienta	database.dart: getBookings() vs getUpcomingBookings(email)	wysokie	UI nie jest granicą zaufania - bez Rules filtr we Flutterze nic nie daje
Klucz Maps / config Firebase w APK	AndroidManifest meta-data, firebase_options.dart	średnie	Binarka jest publiczna; klucz trzeba ograniczyć w konsoli, nie ukrywać w kodzie
Nieużywane uprawnienia lokalizacji	AndroidManifest.xml	średnie	Sklep i audyt pytają po co GPS; zbędna powierzchnia prywatności

Luka / zagrożenie	Gdzie	Ryzyko (niskie / średnie / wysokie)	Dlaczego takie ryzyko
Brak deep link, WebView z whitelistą, biometrii	umowa o powierzchni ataku - jeszcze niezaimplementowane	niskie na dziś	Ryzyko przy implementacji (Z4); Michał: linki/WebView; Jakub: biometria

4. Jak to rozwiązuje nasza platforma?

Jaki mechanizm systemowy lub biblioteka w waszym stacku odpowiada za ten obszar? Jakie są pułapki (domyślne ustawienia, przestarzałe API, różnice między Androidem a iOS)? To jest materiał do porównania między sekcjami na koniec zajęć.

Mechanizm / biblioteka	Do czego go użyjemy	Pułapki i ograniczenia
Firebase Auth + ID token (JWT)	Tożsamość; później ewentualnie custom claims zamiast kolekcji Admin/Employee	Token da się wyciągnąć z pamięci/root. Claims i role weryfikować w Rules, nie tylko we Flutterze
Cloud Firestore / Storage Security Rules + emulator	Autoryzacja: własna rezerwacja vs wszystkie; zakaz zapisu ról z klienta	Na czas rozwoju w konsoli bywa allow read, write: if true. Test bez emulatora kłamie
flutter_secure_storage -> Android Keystore	Zastąpić SharedPreferences dla UID / e-mail / danych sesji	allowBackup; na web Keystore nie istnieje - inny fallback
TLS w SDK Firebase	Poufność na granicy aplikacja <-> sieć	Brak pinningu w SDK z pudełka; proxy + user CA działa na telefonie deweloperskim
permission_handler + App Links / local_auth (plan)	Kamera, galeria, powiadomienia; później deep link i WebView (Michał) oraz biometria (Jakub)	Manifest ma też lokalizację przez Maps. Deep link: intenty z innych aplikacji. Biometria odblokowuje klucz, nie zastępuje logowania Firebase

5. Plan na kolejny tydzień

Każde zadanie musi mieć osobę odpowiedzialną i sposób wykazania, że działa.

Zadanie	Osoba	Jak udowodnimy, że działa	Pozycja w checkliście MASVS
DFD z granicami zaufania (Mermaid + PNG) i utrzymanie	Michał Dyrka	Pliki w docs/threat-model/ (źródło + PNG)	model zagrożeń v1

Zadanie	Osoba	Jak udowodnimy, że działa	Pozycja w checkliście MASVS
docs/threat-model/			
Przegląd i uzupełnienie tabeli STRIDE (min. 10 zagrożeń)	Michał Dyrka	docs/threat-model/threats-v1.md - rozumie każdy wiersz na rundzie	model zagrożeń v1
Audyt uprawnień: GPS w manifeście vs mapa bez lokalizacji użytkownika	Michał Dyrka	Diff AndroidManifest.xml + notatka mapa bez GPS	MASVS-PLATFORM, MASVS-PRIVACY
firestore.rules + storage.rules; testy emulatora klient/pracownik/admin	Mateusz Sarwa	Log: klient nie czyta cudzego Booking; nie zapisuje Admin/{email}	MASVS-AUTH, MASVS-NETWORK
Migracja do flutter_secure_storage; przegląd allowBackup	Jakub Różycki	Po logowaniu brak e-maila w jawnym shared_prefs/*.xml	MASVS-STORAGE-1
Aktualizacja karty (3 os.) w README i zgłoszenie zmiany prowadzącemu	Jakub Różycki	README + karta z Michałem Dyrką	wymóg Z1 / prowadzący

6. Pytania do prowadzącego

- 1. Czy custom claims w Firebase Auth są wymagane, czy wystarczą Security Rules oparte na kolekcjach Admin/Employee (ID dokumentu = e-mail), pod warunkiem że klient nie może tych kolekcji zapisywać?
  2. Aplikacja ma też target web (localhost) - czy MASVS stosujemy formalnie tylko do Androida, a web zostawiamy jako środowisko testów?
  3. Uprawnienie lokalizacji: Google Maps na Androidzie często chce GPS. Czy usunięcie ACCESS_*_LOCATION i zostawienie tylko znacznika salonu jest OK na umowę (3 uprawnienia: kamera, pliki, powiadomienia)?
  4. Do sekcji doszedł Michał Dyrka - czy wystarczy zaktualizowana karta w repo, czy trzeba osobno pisać w sprawie składu?

Notatki prowadzącego

Uwagi z rundy	Ocena aktywności (0-2)

Uwagi z rundy	Ocena aktywności (0-2)